课程目标

我们正式开始，我们今天要学的东西是这些，学完之后你需要掌握到什么程度呢?

1. 掌握Redis的由来以及核心特性
2. 掌握Redis常用5种数据类型以及使用场景

Redis安装

安装gcc依赖

Redis是C语言编写的，编译需要GCC。
Redis6.x.x版本支持了多线程，需要gcc的版本大于4.9，但是CentOS7的默认版本是4.8.5。
查看gcc的版本：

```
gcc -v
```

升级gcc版本：

```
yum -y install centos-release-scl

yum -y install devtoolset-9-gcc devtoolset-9-gcc-c++ devtoolset-9-binutils

scl enable devtoolset-9 bash

echo "source /opt/rh/devtoolset-9/enable" >>/etc/profile
```

确认gcc的版本（在同一个窗口中！）：

```
gcc -v
```

咕泡出品

修改config文件

```
daemonize yes   //更改为后台启动
#bind 127.0.0.1  //注释
protected-mode no  //允许外部网络直接访问
```

启动redis

如果src没有redis-server，在redis目录执行make

```
[root@localhost src]# ./redis-server ../redis.conf   //启动服务

[root@localhost src]# ./redis-cli    //进入客户端
```

RedisCluster集群安装

RedisCluster至少得三主三从 6个节点

6节点分别以127.128.129 3台机器的 6380、6381端口

复制原有安装文件,把原有文件改成6380

```
cp -r redis-6.0.16 redis-6.0.16-6381
mv redis-6.0.16 redis-6.0.16-6380
```

更改配置文件

```
 #bind 127.0.0.1
protected-mode no
daemonize yes
cluster-enabled yes  开启集群
cluster-config-file nodes-6380.conf
pidfile /var/run/redis_6380.pid
```

开启Cluster集群

```
[root@localhost src]# ./redis-cli --cluster create
192.168.8.129:6380 192.168.8.129:6381 192.168.8.128:6381
192.168.8.128:6380 192.168.8.127:6380 192.168.8.127:6381 -
-cluster-replicas 1
```

集群相关命令（需要进入客户端）

cluster info： 打印集群的信息
cluster nodes： 列出集群当前已知的所有节点（node），以及这些节点的相关信息。
cluster meet： 将 ip 和 port 所指定的节点添加到集群当中，让它成为集群的一份子。
cluster forget <node_id>： 从集群中移除 node_id 指定的节点(保证空槽道)。
cluster replicate <node_id>： 将当前节点设置为 node_id 指定的节点的从节点。
cluster saveconfig： 将节点的配置文件保存到硬盘里面。

槽slot命令

cluster addslots [slot ...]： 将一个或多个槽（slot）指派（assign）给当前节点。
cluster delslots [slot ...]： 移除一个或多个槽对当前节点的指派。
cluster flushslots： 移除指派给当前节点的所有槽，让当前节点变成一个没有指派任何槽的节点。
cluster setslot node <node_id>： 将槽 slot 指派给 node_id 指定的节点，如果槽已经指派给另一个节点，那么先让另一个节点删除该槽>，然后再进行指派。
cluster setslot migrating <node_id>： 将本节点的槽 slot 迁移到 node_id 指定的节点中。
cluster setslot importing <node_id>： 从 node_id 指定的节点中导入槽 slot 到本节点。
cluster setslot stable： 取消对槽 slot 的导入（import）或者迁移（migrate）。

键命令

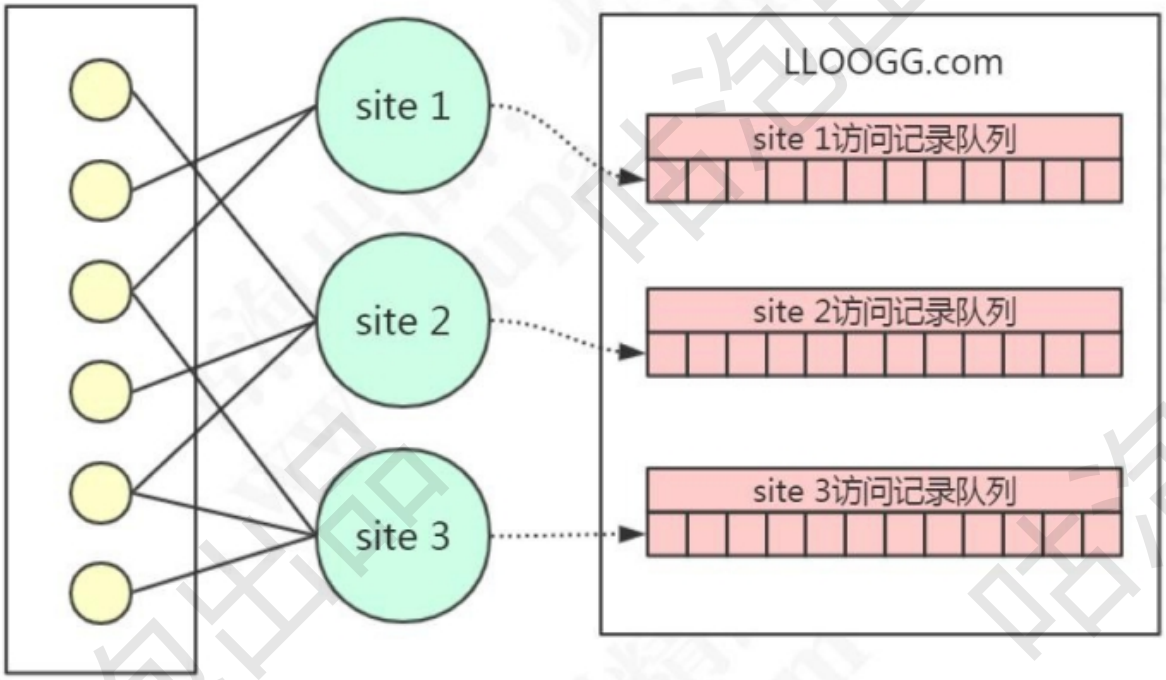
cluster keyslot： 计算键 key 应该被放置在哪个槽上。
cluster countkeysinslot： 返回槽 slot 目前包含的键值对数量。
cluster getkeysinslot： 返回 count 个 slot 槽中的键

核心知识

Redis由来

08年的时候有一个意大利西西里岛的小伙子，笔名antirez（http://invece.org/）,创建了一个访客信息网站 LLOOGG.COM。这个网站是干嘛的呢，其实就是跟我们的百度统计一样！需要知道网站的访问情况，比如访客的IP、操作系统、浏览器、使用的搜索关键词、所在地区、访问的网页地址等等。我们不用自己写代码去实现这个功能，只需要在全局的 footer 里面嵌入一段JS 代码就行了，当页面被访问的时候，就会自动把访客的信息发送到这些网站统计的服务器，然后我们登录后台就可以查看数据了。

LLOOGG.COM 提供的就是这种功能，它可以查看最多10000条的最新浏览记录。 这样的话，它需要为每一个网站创建一个列表（List），不同网站的访问记录进入到不同的列表。如果列表的长度超过了用户指定的长度，它需要把最早的记录删除（先进先出）。



大家来想一下这样的底层数据怎么来保存了！肯定是用数据库对吧！

他最开始的时候用的就是mysql。但是当 LLOOGG.COM 的用户越来越多的时候，它需要维护的列表数量也越来越多，这种记录最新的请求和删除最早的请求的操作也越来越多。

然后mysql数据还是写在磁盘的，那么每一次记录和删除都要读写磁盘，因为数据量和并发量太大，在这种情况下无论怎么去优化数据库都不管用了。

他考虑到最终限制数据库性能的瓶颈在于磁盘，所以打算放弃磁盘，自己去实现一个具有列表结构的数据库的原型，把数据放在内存而不是磁盘，这样就可以大大地提升列表的操作和查的效率。antirez发现这种思路确实能解决这个问题。

所以他用C语言重写了一个内存数据库！

09年，Redis这个内存数据库出世了。从最开始只支持列表的数据类型，到现在支持多种数据类型，并且提供了一系列的高级特性，Redis已经成为一个在全世界被广泛使用的开源项目。这就是redis的由来！

那么redis的全称是全称是REmote DIctionary Service，直接翻译过来是远程字典服务

Redis初识

安装文档 : https://gper.club/articles/7e7e7f7ff2g51gc9g68

官网介绍：https://redis.io/topics/introduction

中文官网：http://www.redis.cn

Redis特性

快 为什么快?

1.基于内存操作，操作不需要跟磁盘交互

2.本身就是k-v结构，redis自己底层数据结构支持，比如跳表

3.命令执行是单线程，同时通信采用多路复用

4.IO多路复用，单个线程中通过记录跟踪每一个sock（I/O流） 的状态来管理多个I/O流 。

其他特性：

- Key-value字典结构存储，存储在内存，key是string类型，value根据不同的数据类型而不同

  它的出世就是为了解决磁盘的性能问题,既然是解决性能问题，所以它是一个以key-value的字典结构存储方式，其中key的最大值为512M，这个数据来自于官网（https://redis.io/topics/data-types-intro/），value根据不同类型有不同的大小https://redis.io/topics/data-types
- 更丰富的数据类型 ，value可以存储很多的数据类型
- 功能丰富：持久化机制、过期策略
- 支持多种编程语言
- 高可用，集群

Redis基本操作get

它默认有16个库（0-15），这个其实可以在配置文件中修改的，默认使用第一个db0。

我们可以进行数据库切换

切换数据库

```
select 0
```

我们刚才讲过它是一个key-value的方式

所以存值

```
set huihui 666
```

取值

```
get huihui
```

我是不是能像数据库的一样每个业务用不同的db?

能是能，但是我不建议！为什么呢，因为我们有个命令能很轻松的删库跑路！

清空所有数据库

```
flushall
```

清空当前数据库

```
flushdb
```

查看所有键

```
keys *
```

获取键总数

```
dbsize
```

咕泡出品

重命名键

```
rename huihui huihui2
```

对某个key设置过期时间

```
expire huihui 10

pexpire huihui 10000
```

查看键是否存在

```
exists huihui
```

删除键

```
del huihui huihui1
```

查看对外类型

```
type huihui2
```

这些是一些比较基本的命令，还有一些命令我就不一一列举了，大家可以自己课后去看下,网址（http://redisdoc.com/index.html）

Redis5大数据类型以及应用场景

官网：https://redis.io/topics/data-types-intro

String、Hash、Set、List、Zset、Hyperloglog（基数统计）、Geo（地理位置）、Streams流。

String

我们刚才已经知道了redis里面都是key-value的结构存储的，那么string数据类型那个value可以存储什么？其实这个value可以存储String,Number,Float,Bits等等.....

那么这个String到底是怎么玩的，我们来看下string的一些基本指令！

基本指令

批量设置

```
mset huihui niubi huihui1 222
```

批量获取

```
mget huihui huihui1
```

获取长度

```
strlen huihui
```

字符串追求内容

```
append huihui good
```

获取指定范围的字符

```
getrange huihui 0 5
```

（整数）值递增

```
incr intkey

incrby intkey 100
```

（浮点数）值递增看

```
set f 2.6

incrbyfloat f 7.3
```

如果不存在这个key设置成功，存在设置失败（分布式锁）

```
set lock1 1 EX 10 NX
```

咕泡出品

EX – 设置指定的到期时间(以秒为单位)。

PX - 设置指定的到期时间(以毫秒为单

NX - 仅在键不存在时设置键。

XX - 只有在键已存在时才设置。

```
setnx k1 1
```

应用场景

计数器：incr 帖子key

读取：mget批量获取

这样我们就能实现一个浏览次数的一个实现！有人会问，老师老师，你这个难道没有线程安全问题嘛？我2个人同时浏览会不会只加1次！

没有的，刚才我们讲过了它的命令执行是单线程的！

```
//region 计算器
for (int i = 0; i <5 ; i++) {
    JedisUtil.getJedisUtil().incr(key: "count: "+i);
//    System.out.println(count);
}
JedisUtil.getJedisUtil().incr(key: "count: 1");
List<String> strList=JedisUtil.getJedisUtil().mget(...keys: "count: 1","count: 2","count: 3");
for (String s:strList) {
    System.out.println(s);
}
//endregion
```

还有分布式锁，锁的概念简单点讲就是我在一段时间内只能有1个人来处理一个事情，我们先看下它的一个总体的实现思路

1. 去获取锁，获取到了就锁定资源
2. 完成业务逻辑后删除分布式锁，删除键
3. 防止意外没有删除锁，每个锁设置过期时间

这样的话，用我们set的哪个命令最适合，setnx，对，没错，就是它，但是我们发现第三步的时候得单独设置，是不是不能保证2个命令的原子性！所以，我们有了后面的set可以加nx的参数！这样我们就能保证获取锁跟设置锁的时间2个操作的原子性!

或者我们可以引入Lua脚本。

Lua脚本是一种轻量级脚本语言，它是用C语言编写的，跟数据的存储过程有点类似。是一种胶水语言，所谓胶水，就是必须要粘合其他东西才能用，要有宿主，它自己不能独立运行

使用 Lua 脚本来执行 Redis 命令的好处：

1、一次发送多个命令，减少网络开销。

2、Redis会将整个脚本作为一个整体执行，不会被其他请求打断，保持原子性。

3、对于复杂的组合命令，我们可以放在文件中，可以实现程序之间的命令集复用。

我们直接来看下在redis里面它是怎么操作的！

redis> eval "redis.call('set',KEYS[1],ARGV[1]) return KEYS[1]" 1 huihui 18

我们再来总结下String能干啥！

1. 缓存 这个我们就不用多讲了，redis用过肯定最先用到的是这个！Set一个值，如果值还在，那么我们就去get就行
2. 分布式ID 大家想下这个ID我们应该用什么命令来实现？ 分布式ID我们知道它要全局唯一！Incr，因为它是单线程下自增的！
3. 计数器、限流！这个我们incrby指令可以决定我们每次增长的长度，那么限流是不是也能做
4. 分布式锁 set 加nx参数 或者setnx （命令执行是单线程的)

好，这个是我们的一个string的一些基本的场景，现在让你去用，你会用并且清楚明白的来刷个6，没明白的刷0，并提出你的问题！

底层数据编码

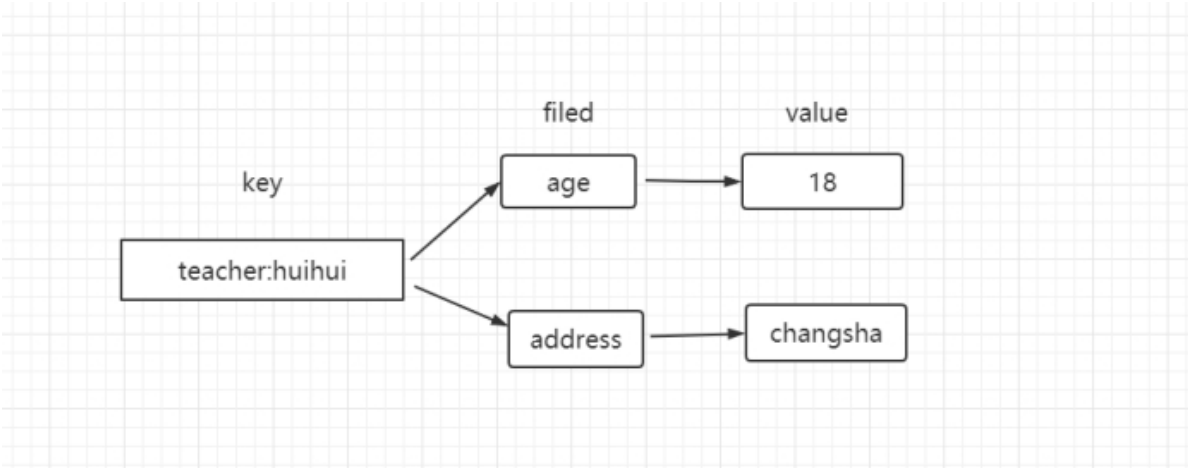
```
object encoding huihui 查看key的编码
```

1. int，存储8个字节的长整型（long，2^63-1）。
2. embstr, 代表embstr格式的SDS，存储小于44个字节的字符串。

3. raw，存储大于44个字节的字符串。

Hash哈希

我们发现我们string可以存浮点，整型，字符串类型，但是假如我要存一个这样的咋办！有人可能会说，老师老师，这个简单，我可以对象json化一下，存到string类型里面就OK了！但是如果我要对某个filed更改下！是不是业务处理就比较麻烦！



所以hash就出现了，都是存的字符串！但是hash有个file。File又是一个key-value!

那么面试题来了，为什么要有这个hash数据类型?

1、把所有相关的值聚集到一个大key中，节省内存空间

2、只使用一个key，减少key冲突

3、当需要批量获取值的时候，只需要使用一个命令，减少内存/IO/CPU 的消耗

但是,它也有一个不足

1.不能设置file的过期时间，我们只能针对于key设置过期时间

基本指令

设置与批量设置

```
hset h1 f 6
hset h1 e 5
hmset h1 a 1 b 2 c 3 d 4
hget h1 a
hmget h1 a b c d
hkeys h1
hvals h1
hgetall h1
hincrby h1 a 10  给某个字段添加值
```

Key操作

```
hexists h1 y  查询key中file是否存在

hdel h1  a

hlen h1
```

使用场景

String可以做的事情，Hash都可以做！为什么这么说，因为hash就比String多了一层key而已！并且单个filed可以单独更改

1. 存储对象类的数据
2. 统计类的数据
3. 购物车

   比如对象或者一张表的数据，比String节省了更多 key 的空间，也更加便于集中管理

   购物车，还可以通过hincrby hkey f 1 加数量！（我们看PPT）

   购物车大家应该很熟悉吧，如果你们家里有个败家娘们的话。。。

咕泡出品

我们知道，世界上最贵的车是购物车对不对，那么这个世界上最贵的车到底是怎么实现的！我们先来看下购物车应该要实现哪些功能，这里写几个重要的功能啊

1. 添加商品数量，我买了1个，再想来个
2. 贼有钱，全选付款
3. 没钱了，移除商品！

好。大家根据这3个想下，我们应该会用到哪些指令！

```
Map<String, String> map = new HashMap<>();
//灰灰的ID是18，买了商品ID为10，数量1个
JedisUtil.getJedisUtil().hset(key: "18", field: "10", value: "1");
//10的商品不错，再来二个
JedisUtil.getJedisUtil().hincrby(key: "18", field: "10", value: 2L);
//11的商品再买个
JedisUtil.getJedisUtil().hset(key: "18", field: "11", value: "1");
//有钱，全选付款
map = JedisUtil.getJedisUtil().hgetall(key: "18");
System.out.println(map.toString());
//发现没钱买11了，把11给删掉
JedisUtil.getJedisUtil().hdel(key: "18", ...fields: "11");
map = JedisUtil.getJedisUtil().hgetall(key: "18");
System.out.println(map.toString());
```

4.分布式锁 key为互斥条件、field为线程信息、value为重入次数

底层数据编码

ZipList 压缩列表，当hash对象同时满足以下两个条件的时候，使用ziplist编码：

1）哈希对象保存的键值对数量 < 512个；

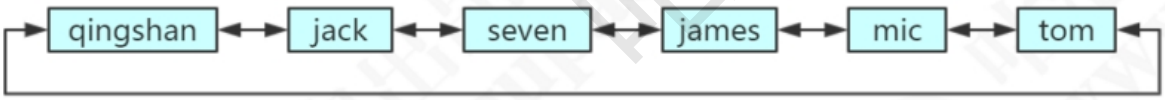
2）所有的键值对的健和值的字符串长度都 < 64byte（一个英文字母一个字节）。

redis.conf配置

```
hash-max-ziplist-value 64 // ziplist中最大能存放的值长度
hash-max-ziplist-entries 512 // ziplist中最多能存放的entry节点数量
```

List列表

基本指令



存储有序的字符串（从左到右），元素可以重复。可以充当队列和栈的角色。

左右添加元素

```
lpush queue a

lpush queue b c
```

右边添加元素

```
Rpush queue d e
```

左右弹出第一条

```
lpop queue

rpop queue
```

那么在弹出命令前面加个b，又是个什么操作，这个就是弹出的时候如果发现没有可以弹出的

左右弹出第一条，并设置超时，直到有数据弹出或者超时

```
blpop queue 10

brpop queue 10
```

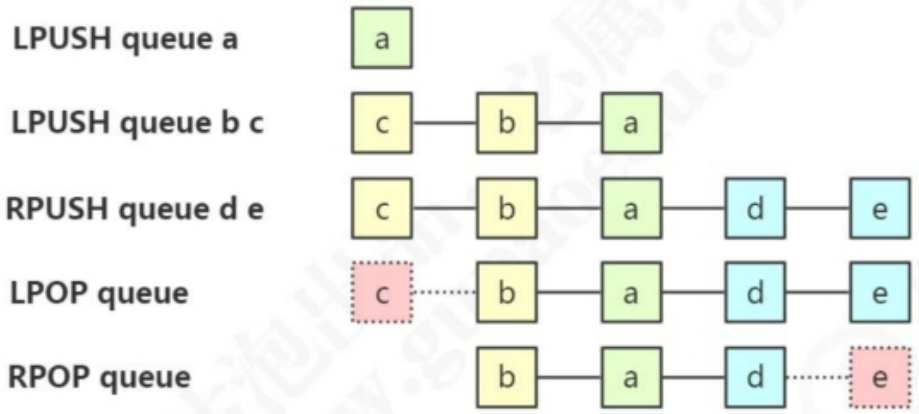
既然是list，我们也要知道怎么取值

取值

```
lindex queue 0

lrange queue 0 -1
```

咕泡出品



应用场景

1. 用户消息时间线

因为list是有序的

```
JedisUtil.getJedisUtil().lpush(key: "msgList", ...strs: "灰灰发送了条消息");
JedisUtil.getJedisUtil().lpush(key: "msgList", ...strs: "lisa发送了个动态");
JedisUtil.getJedisUtil().lpush(key: "msgList", ...strs: "lily发送了个动态");
//左边进左边出（先进后出）
for (int i = 0; i < 3; i++) {
    System.out.println(JedisUtil.getJedisUtil().lpop(key: "msgList"));
}
```

2. 消息队列

List提供了两个阻塞的弹出操作：BLPOP/BRPOP，可以设置超时时间。

BLPOP：BLPOP key1 timeout 移出并获取列表的第一个元素， 如果列表没有元素会阻塞列表直到等待超时或发现可弹出元素为止。

但是队列我不建议大家用redis了，为什么，因为我们有更加成熟的技术，就是我们的MQ！

并且redis它是内存型数据库，可能会造成数据丢失，还有它消费了后没法回应！

队列：先进先出：rpush blpop，左头右尾，右边进入队列，左边出队列。

栈：先进后出：rpush brpop

底层数据编码

quickList 快速列表 quicklist是一个数组+链表的结构

Set集合

String类型的无序集合，最大存储数量 2^32-1（40 亿左右）。

基本指令

添加一个或者多个元素

```
sadd huihuiset a b c d e f g
```

获取所有元素

```
smembers huihuiset
```

获取所有元素的个数

```
scard huihuiset
```

随机获取一个元素

```
srandmember huihuiset
```

随机弹出一个元素

```
spop huihuiset
```

弹出指定元素

```
srem huihuiset g a
```

查看元素是否存在

```
sismember huihuiset e
```

咕泡出品

获取前一个集合有而后面1个集合没有的

```
sdiff huihuiset huihuiset1
```

获取交集

```
sinter huihuiset huihuiset1
```

获取并集

```
sunion huihuiset huihuiset1
```

应用场景

1. 抽奖
2. 点赞、签到等
3. 交集并集 关注等场景

```
//region 关注
//灰灰老师关注的人
JedisUtil.getJedisUtil().sadd( key: "huihuiSub",  ...members: "lily","lisa","zhangsan","lisi");
//lily关注的人
JedisUtil.getJedisUtil().sadd( key: "lilySub",  ...members: "lisa","wangwu","liuliu","liqi");
//
//灰灰跟lily共同关注的人
Set<String> set=JedisUtil.getJedisUtil().sinter( ...keys: "huihuiSub","lilySub");
//灰灰可能认识的人，wangwu.liuliu，liqi都可能认识（第一个有，第二个没有）
Set<String>  set1= JedisUtil.getJedisUtil().sdiff( ...keys: "lilySub","huihuiSub");
System.out.println("灰灰跟lily共同关注的人："+set.toString());
System.out.println("灰灰可能认识的人："+set1.toString());
//endregion
```

```
//region 抽奖
//我们公司有这么多人，有一个是灰灰（重复没用，会覆盖）
//    JedisUtil.getJedisUtil().sadd("gupao","a","b","c","d","e","f","g","h","i","j","k","l",
//            "huihui");
//    //年会抽奖，三等奖3个，二等奖2个，一等奖1个，被抽中的自动从奖池移除
//    List<String> thirdPrizeList=new ArrayList<>();
//    thirdPrizeList.add(JedisUtil.getJedisUtil().spop("gupao"));
//    thirdPrizeList.add(JedisUtil.getJedisUtil().spop("gupao"));
//    thirdPrizeList.add(JedisUtil.getJedisUtil().spop("gupao"));
//    System.out.println("三等奖："+thirdPrizeList.toString());
//
//    //二等奖
//    List<String> secondPrizeList=new ArrayList<>();
//    secondPrizeList.add(JedisUtil.getJedisUtil().spop("gupao"));
//    secondPrizeList.add(JedisUtil.getJedisUtil().spop("gupao"));
//    System.out.println("二等奖："+secondPrizeList.toString());
//
//    //一等奖
//    List<String> firstPrizeList=new ArrayList<>();
//    firstPrizeList.add(JedisUtil.getJedisUtil().spop("gupao"));
//    System.out.println("一等奖："+firstPrizeList.toString());
//endregion
```

底层数据编码

Redis用intset或hashtable存储set。如果元素都是整数类型，就用inset存储。

如果不是整数类型，就用hashtable（数组+链表的存来储结构）。

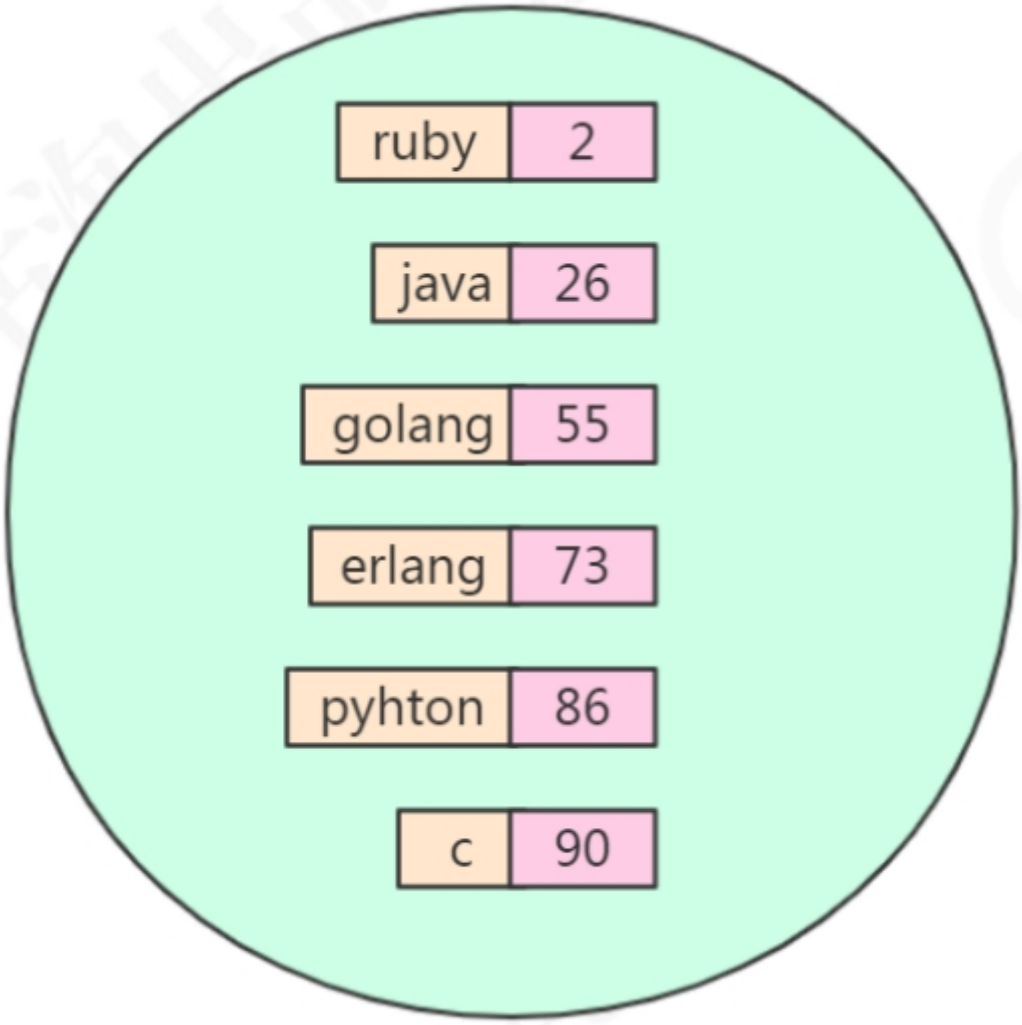
如果元素个数超过512个，也会用hashtable存储。跟一个配置有关：

```
set-max-intset-entries 512
```

问题：set的key没有value，怎么用hashtable存储？value存null就好了。

Sorted Set(ZSet)有序集合

咕泡出品



sorted set，有序的set，每个元素有个score。 我们看张图！

score相同时，按照key的ASCII码排序。

基本指令

批量添加

```
zadd z1 10 a 20 b 30 d 40 c
```

根据分数从低到高

```
zrange z1 0 -1 withscores
```

根据分数从高到低

```
zrevrange z1 0 -1 withscores
```

根据分数范围获取值

```
zrangebyscore z1 20 30
```

移除元素

```
zrem z1 a
```

获取zset个数

```
zcard z1
```

给某个元素加分

```
zincrby z1  20 b
```

获取范围内的个数

```
zcount z1 50 60
```

返回指定元素的索引

```
zrank z1  d
```

获取元素的分数

```
zscore z1 d
```

应用场景

1. 排行榜

咕泡出品

```java
//region 绩效排行
// 现在灰灰公司要做绩效考核 ，默认都是50分
JedisUtil.getJedisUtil().zadd( key: "huihuiCompany",  score: 50,  member: "lily");
JedisUtil.getJedisUtil().zadd( key: "huihuiCompany",  score: 50,  member: "lucy");
JedisUtil.getJedisUtil().zadd( key: "huihuiCompany",  score: 50,  member: "zhangsan");
JedisUtil.getJedisUtil().zadd( key: "huihuiCompany",  score: 50,  member: "lisi");
JedisUtil.getJedisUtil().zadd( key: "huihuiCompany",  score: 50,  member: "wangwu");
JedisUtil.getJedisUtil().zadd( key: "huihuiCompany",  score: 50,  member: "liuliu");

//lisi给灰灰老师买了杯咖啡，加50分
JedisUtil.getJedisUtil().zincrby( key: "huihuiCompany", score: 50, member: "lisi");
//lily长得不错，加20分
JedisUtil.getJedisUtil().zincrby( key: "huihuiCompany", score: 20, member: "lily");
//wangwu 不听话，减10分
JedisUtil.getJedisUtil().zincrby( key: "huihuiCompany", score: -10, member: "wangwu");
//liuliu说灰灰老师坏话，扣20
JedisUtil.getJedisUtil().zincrby( key: "huihuiCompany", score: -20, member: "liuliu");
//张三拍灰灰老师马屁，加10分
JedisUtil.getJedisUtil().zincrby( key: "huihuiCompany", score: 10, member: "zhangsan");
//lucy不上班，开除
JedisUtil.getJedisUtil().zrem( key: "huihuiCompany", ...members: "lucy");
Set<String> set = JedisUtil.getJedisUtil().zrevrange( key: "huihuiCompany",  start: 0,  end: -1);
System.out.println("绩效从高到低："+set);// 从高到低
//endregion
```

这个排行榜一般在公司的使用是在任何排行的地方都可以用到，像销售榜、热搜榜、游戏评分排行等等

底层数据编码

默认使用ziplist编码（第三次见到了，hash的小编码，quicklist的Node，都是ziplist）。

在ziplist的内部，按照score排序递增来存储。插入的时候要移动之后的数据。

如果元素数量大于等于128个，或者任一member长度大于等于64字节使用skiplist+dict存储。

```
zset-max-ziplist-entries 128
zset-max-ziplist-value 64
```

总结

1．我们知道了redis的由来，它是因为意大利西西里岛的一个聪明的小伙子，在做一个访客信息网站时候，为了解决在用mysql的时候带来的性能压力，发明的一个内存型数据库

2．所以就有了它的特性，就是要快

（1）分布在内存以及IO多路复用，速度快

（2）很丰富的数据类型支持

（3）能分布式集群部署，保证高可用

（4）支持持久化，数据过期以及淘汰

我们学习了redis常用的5种数据类型以及使用场景！

（5）String 缓存，计算器，分布式锁等等

（6）Hash 有子键filed 购物车等场景

（7）List 有序集合，元素可以重复 队列，有序列表

（8）Set 无序集合，元素不可重复 关注点赞，抽奖

（9）ZSet 有序集合，元素不可重复 排行榜

咕泡出品